



2

SPIROGRAPHS



You can use a Spirograph toy (shown in [Figure 2-1](#)) to draw mathematical curves. The toy consists of two different-sized rings with plastic gear teeth, one large and one small. The small one has several holes. You put a pen or pencil through one of the holes and then rotate the smaller wheel inside the larger one (which has gear teeth on its inside), keeping the wheels in contact with each other, to draw an endless number of complex and wonderfully symmetric patterns.

In this project, you'll use Python to create an animation of Spirograph-like curves. The program will use parametric equations to describe the motion of a Spirograph's rings and draw the curves (which I call *spiros*). You'll save the completed drawings as PNG image files. Either the program will draw random spiros or you can use command line options to draw a spiro with specific parameters.

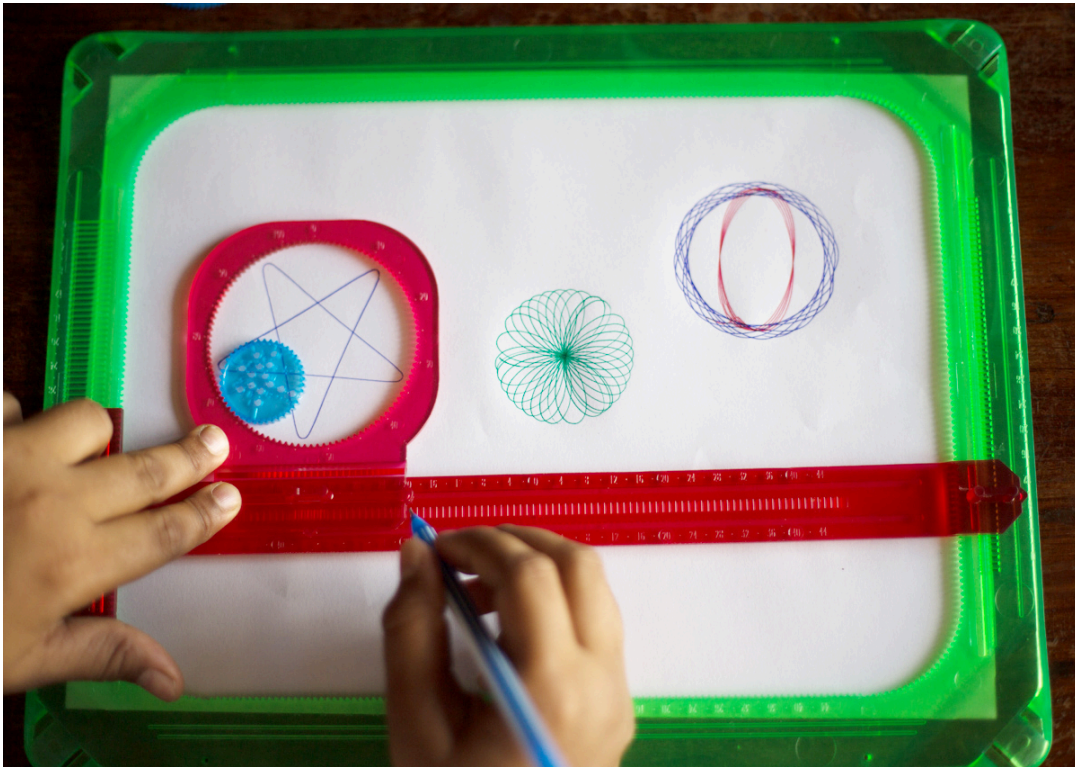


Figure 2-1: A Spirograph toy

In this project, you'll learn how to draw spirograms on your computer. You'll also learn how to do the following:

- Use parametric equations to generate curves.
- Draw a curve as a series of straight lines using the `turtle` module.
- Use a timer to animate graphics.
- Save graphics to image files.

A word of caution: I've chosen to use the `turtle` module to draw spirograms mainly for illustrative purposes and because it's fun, but `turtle` is slow and not ideal for creating graphics when performance is critical. (What do you expect from turtles?) If you want to draw something quickly, there are better ways to do so, and you'll explore some of these options in upcoming projects.

How It Works

This project hinges around using *parametric equations*, equations that express the coordinates of the points on a curve as functions of one or more variables, called *parameters*. You'll plug values for the parameters into equations to calculate points that form a spiro pattern. Then you'll feed those points to the `turtle` module to draw the curves.

Understanding Parametric Equations

To understand how parametric equations work, we'll begin with a simple example: a circle. Consider a circle with radius r , centered at the origin of a two-dimensional plane. That circle consists of all the points whose x- and y-coordinates satisfy the equation $x^2 + y^2 = r^2$. This isn't a parametric equation, however. A parametric equation would give us all possible values of x and y , based on changes in some other variable (the parameter).

Now, consider the following equations:

$$x = r \cos(\theta)$$

$$y = r \sin(\theta)$$

Together, these equations are a *parametric* representation of our circle, where the parameter is θ , the angle of point (x, y) relative to the positive x-axis. Any value of (x, y) in these equations will satisfy the original $x^2 + y^2 = r^2$ equation. As you vary θ from 0 to 2π , the x- and y-coordinates generated by these equations will form the circle. **Figure 2-2** shows this scheme.

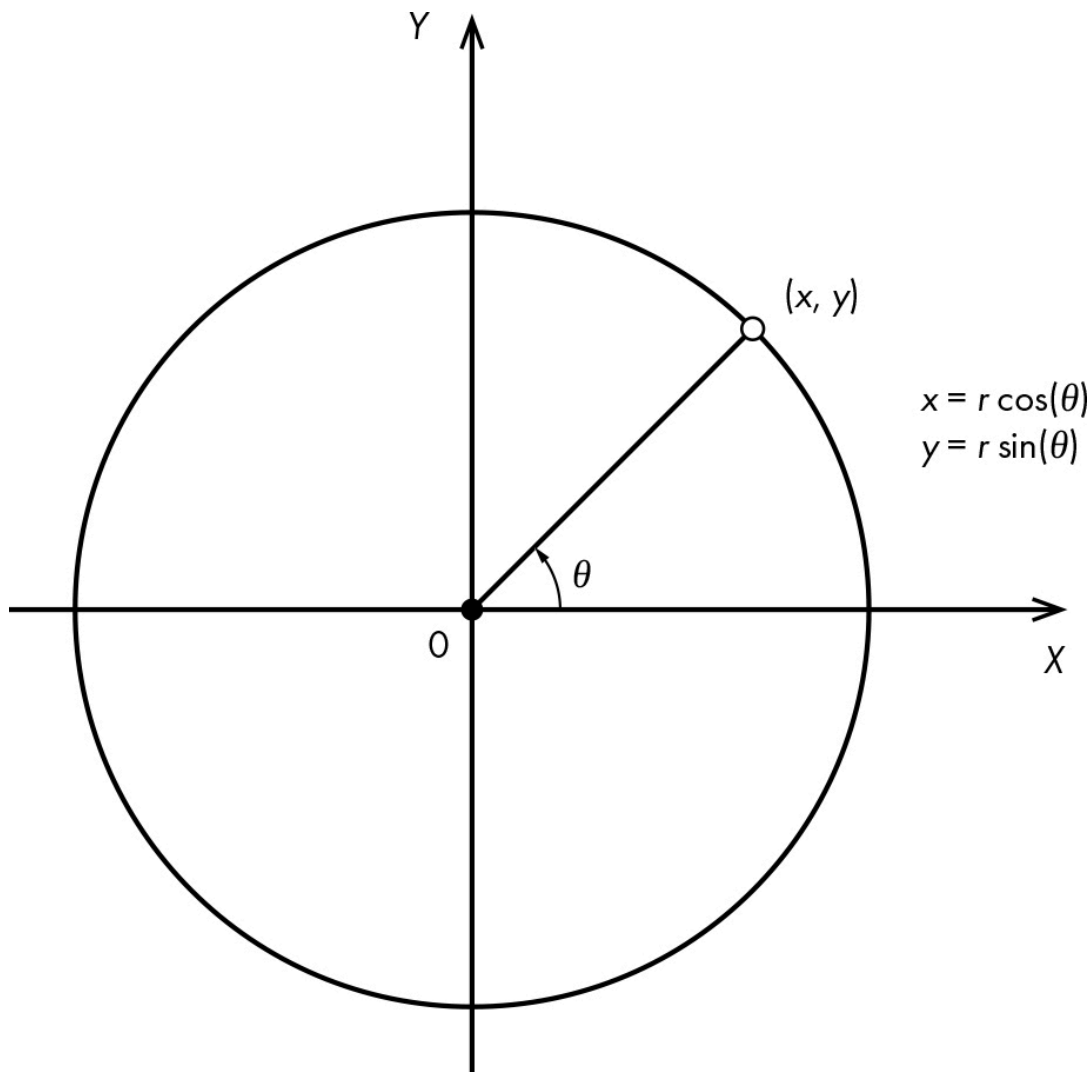


Figure 2-2: Describing a circle with a parametric equation

Remember, these two equations apply to a circle centered at the origin of the coordinate system. You can put a circle at any point in the XY plane by translating the center of the circle from the point (0, 0) to the point (a, b). The more general parametric equations then become $x = a + r \cos(\theta)$ and $y = b + r \sin(\theta)$.

Developing parametric equations that model a Spirograph toy isn't that much different from developing parametric equations for a circle, since at heart a Spirograph simply draws two interlocking circles.

Figure 2-3 shows a mathematical model of Spirograph-like motion. The model has no gear teeth; they're used in Spirograph toys only to

prevent slippage, and in the ideal world of mathematical modeling, you don't have to worry about anything slipping.

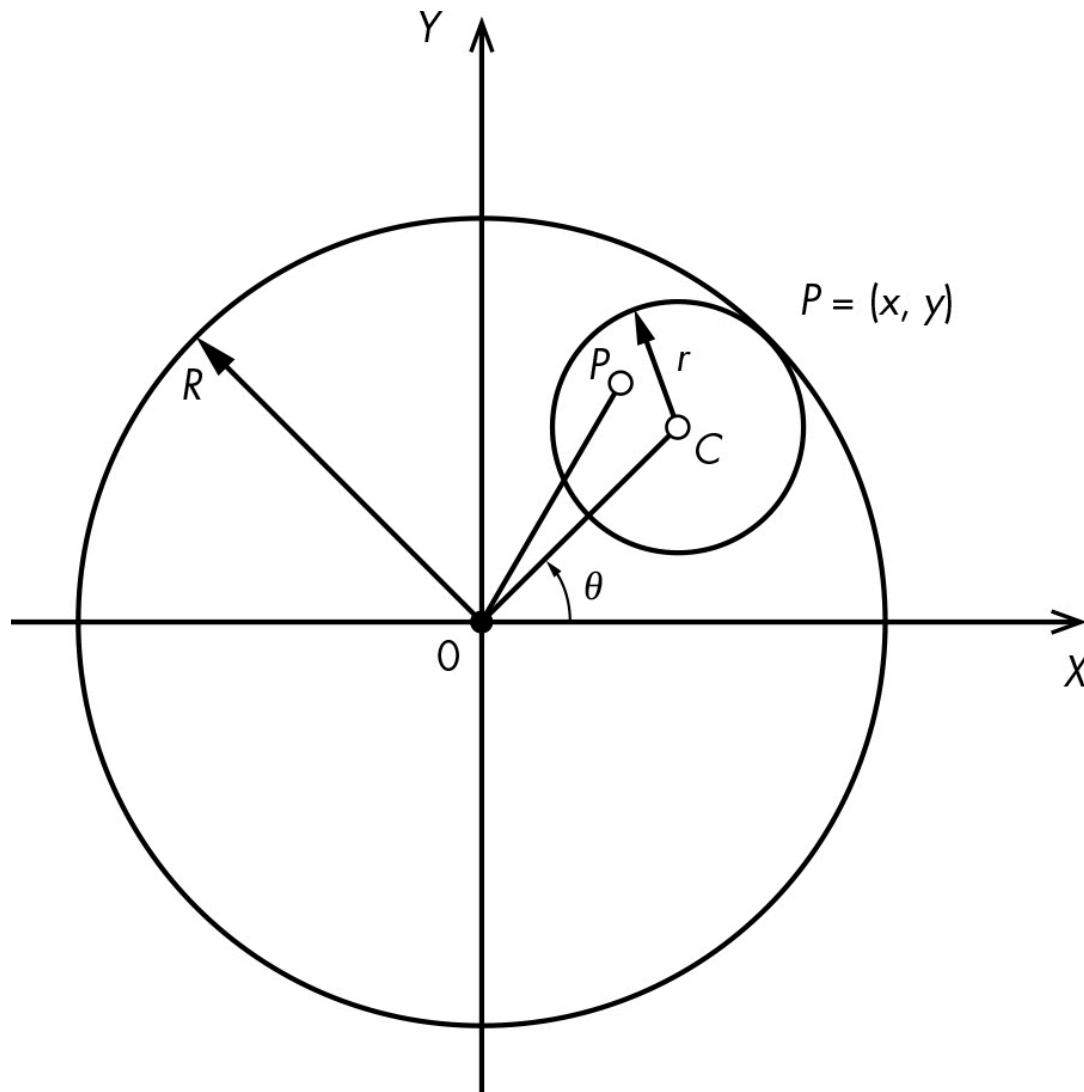


Figure 2-3: A mathematical model of a Spirograph toy

In **Figure 2-3**, C is the center of the smaller circle, P is the pen's tip, and q is the angle of C relative to the positive x-axis. The radius of the bigger circle is R and that of the smaller circle is r . You express the ratio of the radii as variable k , as follows:

$$k = \frac{r}{R}$$

Line segment $\overline{PC}$ tells you how far the pen tip is from the center of the smaller circle. You express the ratio of $\overline{PC}$ to the smaller circle's radius r as the variable l , like so:

$$l = \frac{\overline{PC}}{r}$$

You can now combine these variables into the following parametric equations that represent the x- and y-coordinates of point P (the pen) as the smaller circle rotates inside the larger one:

$$x = R \left((1 - k) \cos (\theta) + lk \cos \left(\frac{1 - k}{k} \theta \right) \right)$$

$$y = R \left((1 - k) \sin (\theta) - lk \sin \left(\frac{1 - k}{k} \theta \right) \right)$$

NOTE *These curves are called hypotrochoids. Although the equations may look a bit scary, the derivation is pretty straightforward. See the Wikipedia page on Spirographs if you'd like to explore the math:*

<http://en.wikipedia.org/wiki/Spirograph>.

Figure 2-4 shows an example curve drawn with these equations. For this curve, I set R to 220, r to 65, and l to 0.8. By choosing different values for these three parameters and then incrementing angle θ , you can produce an endless variety of fascinating curves.

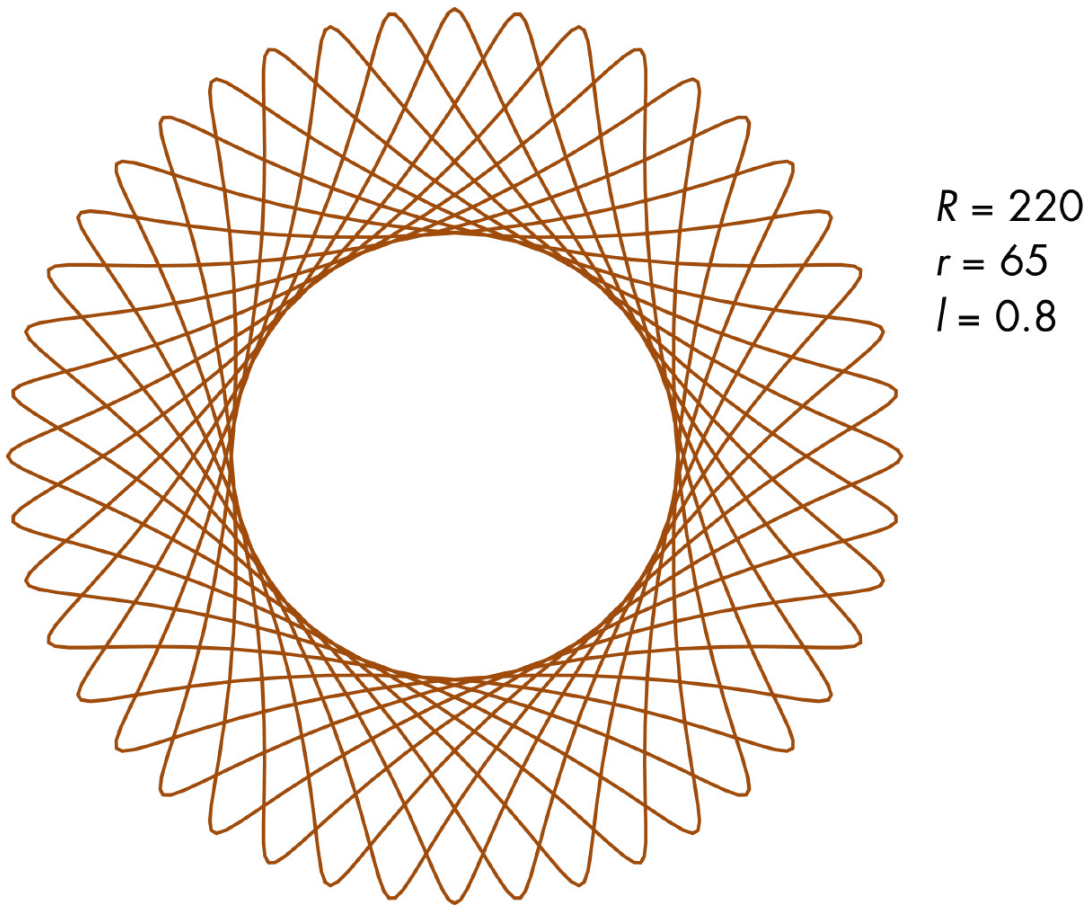


Figure 2-4: A sample curve

The only task left is to determine when to stop drawing, since Spirographs can require many revolutions of the smaller circle around the larger circle to form a complete pattern. You can calculate the *periodicity* of the Spirograph (how long before the Spirograph starts repeating itself) by looking at the ratio of the radii of the inner and outer circles:

$$\frac{r}{R}$$

Reduce this fraction by dividing the numerator and denominator by the *greatest common divisor (GCD)*. Then the numerator tells you how many periods the curve needs to complete itself. For example, in

Figure 2-4, the GCD of (r, R) is 5:

$$\frac{r}{R} = \frac{65}{220} = \frac{(65/5)}{(220/5)} = \frac{13}{44}$$

This tells you that the curve will start repeating itself after 13 revolutions of the smaller circle around the larger circle. The 44 in the denominator tells you the number of times the smaller circle revolves around its own center, which gives you a hint as to the shape of the curve. If you count the petals (or lobes) in the drawing in [Figure 2-4](#), you'll see there are exactly 44!

Once you express the radii ratio in the reduced form r/R , the range for the parameter θ to draw the spiro is $[0, 2\pi r]$. This tells you when to stop drawing a particular spiro. In the case of [Figure 2-4](#), you'd stop when θ reaches 26π (that is, $2\pi \times 13$). Without knowing the ending range of the angle, you'd end up looping around, repeating the curve unnecessarily.

[Drawing Curves with turtle Graphics](#)

Python's `turtle` module doesn't have a method for drawing curved lines. Instead, you'll draw a spiro as a collection of straight lines between different points calculated with the parametric equations discussed in the previous section. As long as the change in angle θ from one point to the next is relatively small, the result will appear curved.

To demonstrate, the following program draws a circle with `turtle`. It uses our basic parametric equations for a circle, $x = a + r \cos(\theta)$ and $y = b + r \sin(\theta)$, to calculate points along the circle, and it connects those points with straight lines. Technically, the program actually produces an N -sided polygon, but because the angle parameter will change in small increments, N will be very large, and the polygon will look like a circle. Enter the following code, save it as *drawcircle.py*, and run it in Python:


```
import math

import turtle

# draw the circle using turtle

def drawCircleTurtle(x, y, r):

    # move to the start of circle

    turtle.up()

    ❶ turtle.setpos(x + r, y)

    turtle.down()

    # draw the circle

    ❷ for i in range(0, 365, 5):

        ❸ a = math.radians(i)

        ❹ turtle.setpos(x + r*math.cos(a), y + r*math.sin(a))

    ❺ drawCircleTurtle(100, 100, 50)

turtle.mainloop()
```

Here you define the `drawCircleTurtle()` function, whose parameters are the center of the circle to be drawn, (x, y) , and the circle's radius, r . The function starts by moving the turtle into position at the first point on the circle's horizontal axis: $(x + r, y)$ ❶. The calls to `up()` and `down()` prevent the turtle from drawing while it's getting into position. Next, you start a loop using `range(0, 365, 5)`, which increments the variable `i` in steps of 5 from 0 to 360 ❷. The `i` variable is the angle parameter you'll pass into the parametric circle equations,

but first you convert it from degrees to radians ③. (Most computer programs require radians for angle-based calculations.)

Compute the next set of circle coordinates using the two parametric equations, and you set the turtle's position accordingly ④. This draws a straight line from the last turtle position to the newly calculated one. Since you're changing the angle parameter by just 5 degrees at a time, the straight lines will create the appearance of a round circle.

Now that you have your function, you call it to draw a circle ⑤. Calling `turtle.mainloop()` keeps the `tkinter` window open so that you can admire your work. (`tkinter` is the default GUI library used by Python.)

You're now ready to draw some spirographs! You'll use the same `turtle` approach illustrated earlier. All that has to change are the details of the parametric equations used to calculate the points.

Requirements

You'll use the following to create your spirographs:

- The `turtle` module for drawing
- `Pillow`, a fork of the *Python Imaging Library (PIL)*, to save the spirograph images

The Code

First you'll define a class `Spiro` to draw the curves. You can use this class either to draw a single curve with customizable parameters or as part of an animation that draws several random spirographs concurrently. To coordinate the animation, you'll define another class called `SpiroAnimator`. At the top level of the program, you'll write a function to save your drawings as image files, and you'll use the `main()` function to take in user input and set the drawing in motion.

To see the full project code, skip ahead to **[“The Complete Code”](#)** on **[page 36](#)**. You can also download the code for this project from **<https://github.com/mkvenkit/pp2e/blob/main/spirograph/spiro.py>**.

Drawing the Spiros

The `Spiro` class features methods for drawing an individual spiro pattern. Here’s the `Spiro` class’s constructor:

```
class Spiro:
```

```
    # constructor
```

```
    def __init__(self, xc, yc, col, R, r, l):
```

```
        # create the turtle object
```

```
        ❶ self.t = turtle.Turtle()
```

```
        # set the cursor shape
```

```
        ❷ self.t.shape('turtle')
```

```
        # set the step in degrees
```

```
        ❸ self.step = 5
```

```
        # set the drawing complete flag
```

```
        ❹ self.drawingComplete = False
```

```
        # set the parameters
```

```
        self.setparams(xc, yc, col, R, r, l)
```

```
        # initialize the drawing
```

```
self.restart()
```

The `Spiro` constructor creates a new `turtle` object ❶. This way, each individual `Spiro` object will have its own `turtle` object associated with it, meaning you can create multiple `Spiro` objects to draw a bunch of spiros simultaneously. You set the shape of the turtle cursor to a turtle ❷. (You'll find other choices in the `turtle` documentation at <https://docs.python.org/3/library/turtle.html>.) You set the angle increment for the parametric drawing to 5 degrees ❸ and create a Boolean `drawingComplete` flag for indicating when the spiro is done ❹. This flag will be useful when multiple `Spiro` objects are drawing in tandem; it allows you to keep track of whether a particular spiro is complete. You finish the constructor by calling two setup methods, as discussed next.

The Setup Methods

The `Spiro` class's `setparams()` and `restart()` methods both help with some setup that needs to be done before a spiro pattern can be drawn. Let's look at the `setparams()` method first:

```
def setparams(self, xc, yc, col, R, r, l):
```

```
    # the Spirograph parameters
```

```
    self.xc = xc
```

```
    self.yc = yc
```

```
    self.R = int(R)
```

```
    self.r = int(r)
```

```
    self.l = l
```

```
    self.col = col
```

reduce r/R to its smallest form by dividing with the GCD

❶ `gcdVal = math.gcd(self.r, self.R)`

❷ `self.nRot = self.r//gcdVal`

get ratio of radii

`self.k = r/float(R)`

set the color

`self.t.color(*col)`

store the current angle

❸ `self.a = 0`

First you store the coordinates of the center of the spiro (`xc` and `yc`). Then you convert the radius of each circle (`R` and `r`) to an integer and store the values. You also store `l`, which defines the position of the pen, and `col`, which determines the spiro's color. Next, you use the `gcd()` method from Python's built-in `math` module to compute the GCD of the radii ❶. You use this information to determine the periodicity of the curve, which you save as `self.nRot` ❷. Finally, you set the starting value of the angle parameter, `a`, to `0` ❸.

The `restart()` method continues with the setup by resetting the drawing parameters for the `Spiro` object and getting it into position to draw a spiro. This method makes it possible to reuse the same `Spiro` object to draw multiple spiros, one after the other, as part of the program's animation. The program will call `restart()` each time the object is ready to draw a new spiro. Here's the method:

```
def restart(self):
```

```
# set the flag
```

```
self.drawingComplete = False
```

```
# show the turtle
```

```
self.t.showturtle()
```

```
# go to the first point
```

```
❶ self.t.up()
```

```
❷ R, k, l = self.R, self.k, self.l
```

```
a = 0.0
```

```
❸ x = R*((1-k)*math.cos(a) + l*k*math.cos((1-k)*a/k))
```

```
y = R*((1-k)*math.sin(a) - l*k*math.sin((1-k)*a/k))
```

```
❹ self.t.setpos(self.xc + x, self.yc + y)
```

```
❺ self.t.down()
```

You reset the `drawingComplete` flag to `False`, indicating the object is ready to draw a new spiro. Then you show the turtle cursor, in case it was hidden. Next lift up the pen ❶ so you can move to the first position at ❹ without drawing a line. At ❷, you're just using some local variables to keep the code compact. Then you feed those variables to the spiro parametric equations to compute the x- and y-coordinates of the curve's starting point, using `0` as an initial value for angle `a` ❸. Finally, once the turtle is in place, you set the pen down so the turtle can start drawing the spiro ❺.

The draw() Method

If you use command line options to set the parameters of a spiro, the program will draw just that one spiro, using the `Spiro` class's `draw()` method. This method draws the entire spiro all in one go, as a continuous sequence of straight-line segments:

```
def draw(self):

    # draw the rest of the points

    R, k, l = self.R, self.k, self.l

    ❶ for i in range(0, 360*self.nRot + 1, self.step):

        a = math.radians(i)

        ❷ x = R*((1-k)*math.cos(a) + l*k*math.cos((1-k)*a/k))

        y = R*((1-k)*math.sin(a) - l*k*math.sin((1-k)*a/k))

        try:

            ❸ self.t.setpos(self.xc + x, self.yc + y)

        except:

            print("Exception, exiting.")

            exit(0)

    # drawing is now done so hide the turtle cursor

    ❹ self.t.hideturtle()
```

Here you iterate through the complete range of the parameter `i`, which is expressed in degrees as 360 times `nRot` ❶. You use the parametric equations to compute the x- and y-coordinates for each value of the `i` parameter ❷, calling the turtle's `setpos()` method ❸ to draw

the line from one point to the next. This method is enclosed in a `try` block so that if an exception arises—such as the user closing the window in the middle of the drawing process—you can catch it and exit gracefully. Finally, you hide the cursor because you’ve finished drawing ④.

The `update()` Method

If you don’t use any command line options, the program will draw multiple random spirographs as an animation. This approach requires some restructuring of the drawing code we just looked at. Instead of drawing an entire spiro in one go, you need a method that draws just a single-line segment of the spiro. Then you’ll call that method at every time step in the animation. This `update()` method of the `Spiro` class fits the bill:

```
def update(self):
```

```
    # skip the rest of the steps if done
```

```
    ❶ if self.drawingComplete:
```

```
        return
```

```
    # increment the angle
```

```
    ❷ self.a += self.step
```

```
    # draw a step
```

```
    R, k, l = self.R, self.k, self.l
```

```
    # set the angle
```

```
    ❸ a = math.radians(self.a)
```

```
x = self.R*((1-k)*math.cos(a) + l*k*math.cos((1-k)*a/k))
```

```
y = self.R*((1-k)*math.sin(a) - l*k*math.sin((1-k)*a/k))
```

```
try:
```

```
    ❷ self.t.setpos(self.xc + x, self.yc + y)
```

```
except:
```

```
    print("Exception, exiting.")
```

```
    exit(0)
```

```
# if drawing is complete, set the flag
```

```
    ❸ if self.a >= 360*self.nRot:
```

```
        self.drawingComplete = True
```

```
# drawing is now done so hide the turtle cursor
```

```
self.t.hideturtle()
```

You first check to see whether the `drawingComplete` flag is set ❶; if not, you continue through the rest of the code. You increment the current angle ❷, calculate the (x, y) position corresponding to the current angle ❸, and move the turtle there, drawing the line segment in the process ❹. This is just like the code inside the `for()` loop in the `draw()` method, except it happens only once.

When I discussed the Spirograph parametric equations, I talked about the periodicity of the curve. A Spirograph starts repeating itself after a certain angle. You finish the `update()` function by checking whether the angle has reached the full range computed for this particular curve ❺. If so, you set the `drawingComplete` flag because the spiro is

finished. Finally, you hide the turtle cursor so you can see your beautiful creation.

Coordinating the Animation

The `SpiroAnimator` class will let you draw several random spiros simultaneously as an animation. This class coordinates the activity of multiple `Spiro` objects with randomly assigned parameters, using a timer to regularly call each `Spiro` object's `update()` method. This technique updates the graphics periodically and lets the program process events such as button presses, mouse clicks, and so on.

Let's look at the `SpiroAnimator` class's constructor first:

```
class SpiroAnimator:
```

```
    # constructor
```

```
    def __init__(self, N):
```

```
        # set the timer value in milliseconds
```

```
        ❶ self.deltaT = 10
```

```
        # get the window dimensions
```

```
        ❷ self.width = turtle.window_width()
```

```
        self.height = turtle.window_height()
```

```
        # restarting
```

```
        ❸ self.restarting = False
```

```
        # create the Spiro objects
```

```
self.spiros = []

for i in range(N):

    # generate random parameters

    ❷ rparams = self.genRandomParams()

    # set the spiro parameters

    ❸ spiro = Spiro(*rparams)

    self.spiros.append(spiro)

# call timer

    ❹ turtle.ontimer(self.update, self.deltaT)
```

The `SpiroAnimator` constructor sets `deltaT` to `10`, which is the time interval in milliseconds you'll use for the timer ❶. You then store the dimensions of the turtle window ❷ and initialize a flag that will be used to indicate that a restart is in progress ❸. In a loop that repeats N times (N is passed into `SpiroAnimator` in the constructor), you create new `Spiro` objects ❹ and add them to a `spiros` list. Before creating each `Spiro` object, you call the `genRandomParams()` helper method ❺ to randomly assign the spiro's parameters (we'll look at this method next). The `rparams` here is a tuple that you need to pass into the `Spiro` constructor. However, the constructor expects multiple arguments, so you use the Python `*` operator to unpack the tuple into a series of arguments. Finally, you set the `turtle.ontimer()` method to call `update()` after `deltaT` milliseconds ❻, which sets the animation in motion.

Generating Random Parameters

You'll use the `genRandomParams()` method to generate random parameters to send to each `Spiro` object as it's created to produce a wide variety of curves. You'll also call this method each time a `Spiro` object finishes drawing a spiro and is ready to start drawing a new one:

```
def genRandomParams(self):  
  
    width, height = self.width, self.height  
  
    R = random.randint(50, min(width, height)//2)  
  
    r = random.randint(10, 9*R//10)  
  
    l = random.uniform(0.1, 0.9)  
  
    xc = random.randint(-width//2, width//2)  
  
    yc = random.randint(-height//2, height//2)  
  
    col = (random.random(),  
  
          random.random(),  
  
          random.random())  
  
    ❶ return (xc, yc, col, R, r, l)
```

To generate random numbers, you use three methods from Python's `random` module: `randint()`, which returns random integers in the specified range; `uniform()`, which does the same for floating-point numbers; and `random()`, which returns a float between 0 and 1. You set `R` to a random integer between 50 and the value of half the smallest dimension of your window, and you set `r` to between 10 percent and 90 percent of `R`. Then you set `l` to a random fraction between 0.1 and 0.9.

You next select a random point on the screen to place the center of the spiro by selecting random x- and y-coordinates (`xc` and `yc`) from within the screen boundaries. You assign a random color `col` to the curve by setting random values for the red, green, and blue color components (these are defined on a scale from 0 to 1). Finally, all of your calculated parameters are returned as a tuple ❶.

Restarting the Animation

The `SpiroAnimator` class has its own `restart()` method for restarting the animation to draw a new set of spiro:

```
def restart(self):

    # ignore restart if already in the middle of restarting

    ❶ if self.restarting:

        return

    else:

        self.restarting = True

    for spiro in self.spiros:

        # clear

        spiro.clear()

        # generate random parameters

        rparams = self.genRandomParams()

        # set the spiro parameters

        spiro.setparams(*rparams)
```

```
# restart drawing
```

```
spiro.restart()
```

```
# done restarting
```

```
❷ self.restarting = False
```

This method loops through all the `Spiro` objects. For each one, you clear the previous drawing and randomly generate a new set of spiro parameters. Then you use the `Spiro` object's setup methods, `set-params()` and `restart()`, to assign the new parameters and get the object ready to draw the next spiro. The `self.restarting` flag ❶ prevents this method from being called before it's had a chance to complete, which could happen if the user repeatedly hits the spacebar. The flag is reset at the end of the method so that the next restart won't be ignored ❷.

Updating the Animation

The following code shows the `update()` method in `SpiroAnimator`, which is called by the timer every 10 milliseconds to update all the `Spiro` objects used in the animation:

```
def update(self):
```

```
    # update all spiros
```

```
    ❶ nComplete = 0
```

```
    for spiro in self.spiros:
```

```
        # update
```

```
        ❷ spiro.update()
```

```
# count completed spiros

❸ if spiro.drawingComplete:

    nComplete += 1

# restart if all spiros are complete

❹ if nComplete == len(self.spiros):

    self.restart()

# call the timer

try:

    ❺ turtle.ontimer(self.update, self.deltaT)

except:

    print("Exception, exiting.")

    exit(0)
```

The `update()` method uses a counter `nComplete` to track the number of `Spiro` objects that have finished drawing ❶. The method loops through the list of `Spiro` objects and updates them ❷, which draws one more line segment in each spiro. You increment the counter if a `Spiro` is done ❸.

Outside the loop, you check the counter to determine whether all the objects have finished drawing ❹. If so, you restart the animation with fresh spiros by calling the `restart()` method. The `update()` method ends with a call to the `turtle` module's `ontimer()` method ❺, which calls `update()` again after `deltaT` milliseconds. This is what keeps the animation going.

Showing or Hiding the Cursor

You use the following method of the `SpiroAnimator` class to toggle the turtle cursors on and off. Turning them off makes the drawing go more quickly.

```
def toggleTurtles(self):  
  
    for spiro in self.spiros:  
  
        if spiro.t.isvisible():  
  
            spiro.t.hideturtle()  
  
        else:  
  
            spiro.t.showturtle()
```

This method uses built-in `turtle` methods to hide the cursor if it's visible or to show the cursor if it isn't. Later, you'll see how this `toggleTurtles()` method is triggered by keypresses while the animation is running.

Saving the Curves

After all your hard work generating spiros, it would be nice to have a way to save the results. The stand-alone `saveDrawing()` function saves the contents of the drawing window as a PNG image file:

```
def saveDrawing():  
  
    # hide the turtle cursor  
  
    ❶ turtle.hideturtle()  
  
    # generate unique filenames
```

```
❷ dateStr = (datetime.now()).strftime("%d%b%Y-%H%M%S")

fileName = 'spiro-' + dateStr

print('saving drawing to {}.eps/png'.format(fileName))

# get the tkinter canvas

canvas = turtle.getcanvas()

# save the drawing as a postscript image

❸ canvas.postscript(file = fileName + '.eps')

# use the Pillow module to convert the postscript image file to
PNG

❹ img = Image.open(fileName + '.eps')

❺ img.save(fileName + '.png', 'png')

# show the turtle cursor

turtle.showturtle()
```

You hide the turtle cursors so that you won't see them in the final drawing ❶. Then you use `datetime()` to generate unique, timestamp-based names for the image files (in the *day-month-year-hour-minute-second* format) ❷. You append this string to *spiro-* to generate the filename.

The `turtle` program uses user interface (UI) windows created by `tkinter`, and you use the `canvas` object of `tkinter` to save the window in the Embedded PostScript (EPS) file format ❸. Because EPS is vector based, you can use it to print your images at high resolution, but the

PNG format is more versatile, so you use `Pillow` to open the EPS file ④ and save it as a PNG file ⑤. Finally, you unhide the turtle cursors.

Parsing Command Line Arguments and Initialization

Most projects in this book have command line arguments for customizing the code. Rather than trying to parse them by hand and creating a mess, delegate this mundane task to Python's `argparse` module. That's what you do in the first part of the spiro program's `main()` function:

```
def main():
```

```
    ❶ parser = argparse.ArgumentParser(description=descStr)
```

```
    # add expected arguments
```

```
    ❷ parser.add_argument('--sparams', nargs=3, dest='sparams',
required=False,
```

```
                        help="The three arguments in sparams: R, r, l.")
```

```
    # parse args
```

```
    ❸ args = parser.parse_args()
```

You create an `ArgumentParser` object to manage the command line arguments ❶. Then you add the `--sparams` argument to the parser ❷. It consists of three components, for the *R*, *r*, and *l* parameters of a spiro. You use the `dest` option to specify the variable name the values should be stored under once the arguments are parsed, and `required=False` means this argument is optional. You call the `parse_args()` method ❸ to actually parse the arguments. This makes the arguments available as properties of the `args` object. In this case, the values of the `--sparams` argument will be available through `args.sparams`.

NOTE *You'll follow the same basic pattern described here throughout the book to create and parse each project's command line arguments.*

The `main()` function continues by setting up some `turtle` parameters:

```
# set the width of the drawing window to 80 percent of the screen width
```

```
❶ turtle.setup(width=0.8)
```

```
# set the cursor shape to turtle
```

```
turtle.shape('turtle')
```

```
# set the title to Spirographs!
```

```
turtle.title("Spirographs!")
```

```
# add the key handler to save our drawings
```

```
❷ turtle.onkey(saveDrawing, "s")
```

```
# start listening
```

```
❸ turtle.listen()
```

```
# hide the main turtle cursor
```

```
❹ turtle.hideturtle()
```

You use `setup()` to set the width of the drawing window to 80 percent of the screen width ❶. (You could also give `setup()` specific height and origin parameters.) Then you set the cursor shape to a turtle and set the title of the program window to *Spirographs!* Next, you use `onkey()` with your `saveDrawing()` function to instruct the

program to save the drawing when you press the S key on your keyboard ❷. Calling `listen()` makes the drawing window listen for user events (like keypresses) ❸. Finally, you hide the turtle cursor ❹.

The rest of the `main()` function proceeds as follows:

```
# check for any arguments sent to --sparams and draw the  
Spirograph
```

```
❶ if args.sparams:
```

```
    ❷ params = [float(x) for x in args.sparams]
```

```
    # draw the Spirograph with the given parameters
```

```
    col = (0.0, 0.0, 0.0)
```

```
    ❸ spiro = Spiro(0, 0, col, *params)
```

```
    ❹ spiro.draw()
```

```
else:
```

```
    # create the animator object
```

```
    ❺ spiroAnim = SpiroAnimator(4)
```

```
    # add a key handler to toggle the turtle cursor
```

```
    turtle.onkey(spiroAnim.toggleTurtles, "t")
```

```
    # add a key handler to restart the animation
```

```
    turtle.onkey(spiroAnim.restart, "space")
```

```
# start the turtle main loop
```

⑥ turtle.mainloop()

You first check whether any arguments were given to `--sparams` ①; if so, the program will just draw the one spiro defined by those arguments. The arguments currently exist as strings, but you need them to be interpreted as numbers. You use a list comprehension to convert them into a list of floats ②. (A *list comprehension* is a Python construct that lets you create a list in a compact and powerful way. For example, `a = [2*x for x in range(1, 5)]` creates a list of the first four even numbers.) Then you use the parameters to construct a `Spiro` object ③ (with the help of the Python `*` operator, which unpacks the list into a series of arguments) and call `draw()` to draw the spiro ④.

If no arguments were specified at the command line, you enter random animation mode. For this, you create a `SpiroAnimator` object ⑤, passing it the argument `4`, which tells it to draw four spiros at once. Then you use two `onkey` calls to capture additional keypresses. Pressing the T key will show or hide the turtle cursors with the `toggleTurtles()` method, while pressing the spacebar (`space`) will call `restart()` to interrupt the animation at any point and start drawing four different random spiros. Finally, you call `mainloop()` to tell the `tkinter` window to stay open, listening for events ⑥.

Running the Spirograph Animation

Now it's time to run your program:

```
$ python spiro.py
```

By default, the `spiro.py` program draws four random spiros simultaneously, as shown in **Figure 2-5**. Pressing S saves the drawing, pressing T toggles the cursors, and pressing the spacebar restarts the animation.

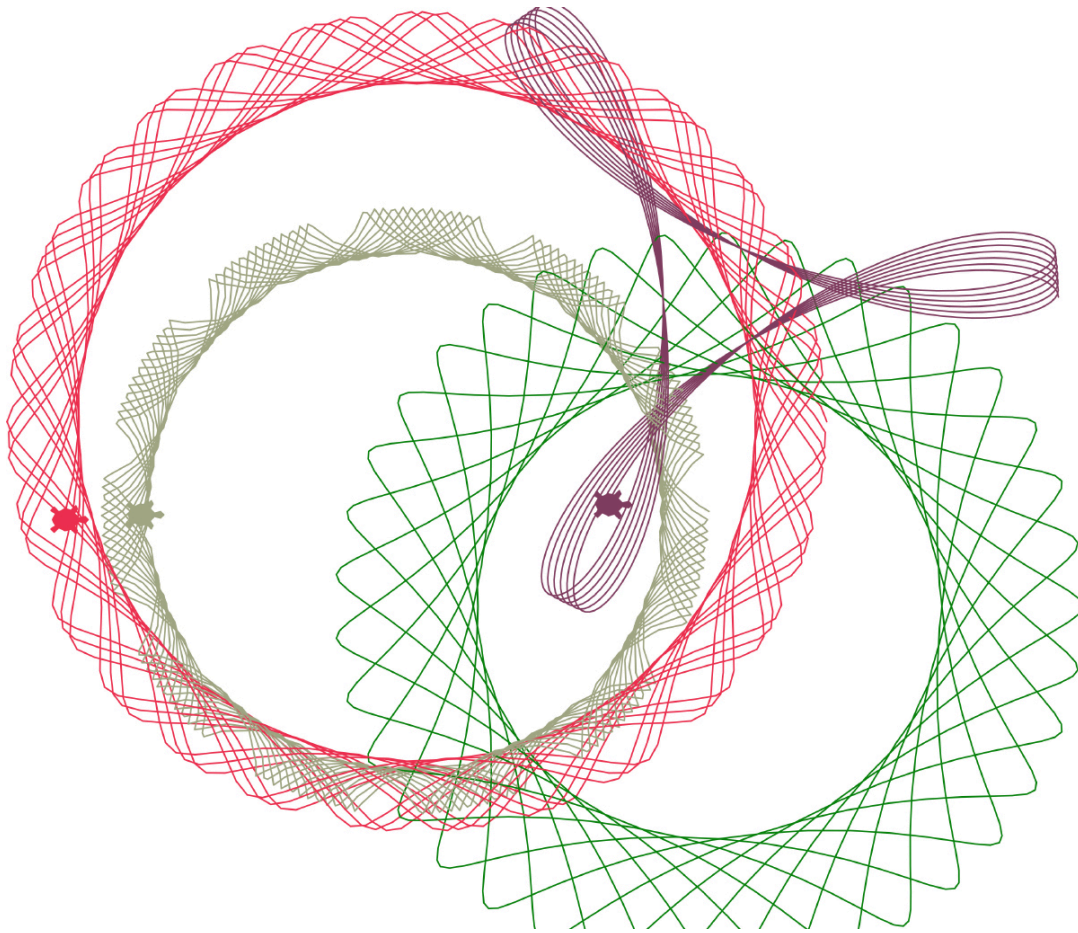


Figure 2-5: A sample run of spiro.py

Now run the program again, this time passing in parameters at the command line to draw a particular spiro:

```
$ python spiro.py --sparams 300 100 0.9
```

Figure 2-6 shows the output. As you can see, this code draws a single spiro with the parameters specified by the user, in contrast to **Figure 2-5**, which displays an animation of several random spiros.

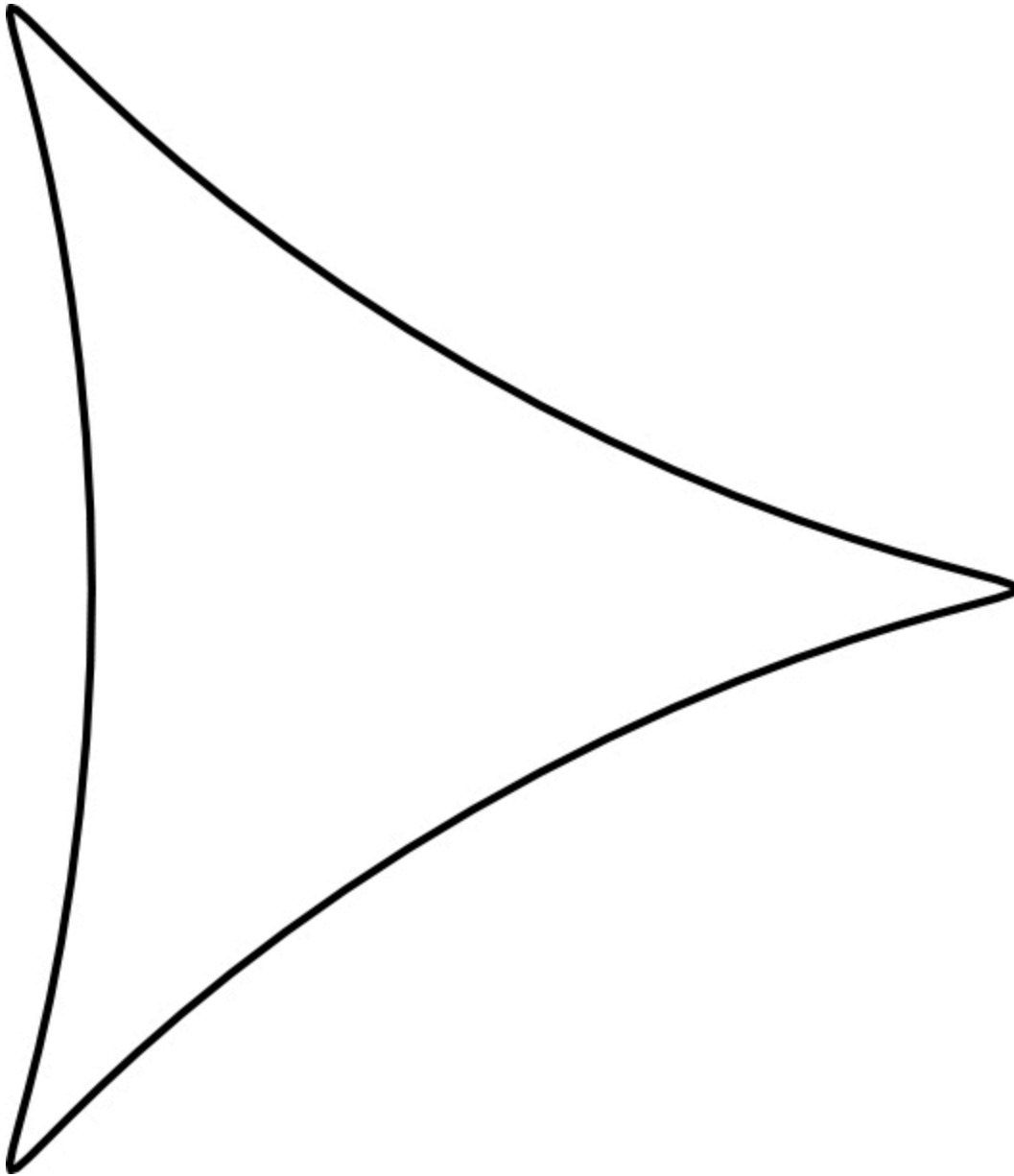


Figure 2-6: A sample run of spiro.py with specific parameters

Have fun experimenting with different parameters to see how they influence the resulting curves.

Summary

In this project, you learned how to create Spirograph-like curves. You also learned how to adjust the input parameters to generate a variety of different curves and to animate them on the screen. I hope you en-

joy creating these spirographs. (You'll find a surprise in [Chapter 13](#), where you'll learn how to project spirographs onto a wall!)

Experiments!

Here are some ways to experiment further with spirographs:

1. Now that you know how to draw circles, write a program to draw random *spirographs*. Find the equation for a *logarithmic spiral* in parametric form and then use it to draw the spirals.
2. You might have noticed that the turtle cursor is always oriented to the right as the curves are drawn, but that's not how turtles move! Orient the turtle so that, as the curve is being drawn, it faces in the direction of drawing. (Hint: calculate the direction vector between successive points for every step and reorient the turtle using the `turtle.setheading()` method.)

The Complete Code

Here's the complete Spirograph program:

```
"""
```

```
spiro.py
```

A Python program that simulates a Spirograph.

Author: Mahesh Venkitachalam

```
"""
```

```
import random, argparse
```

```
import numpy as np
```

```
import math

import turtle

import random

from PIL import Image

from datetime import datetime

# a class that draws a spiro

class Spiro:

    # constructor

    def __init__(self, xc, yc, col, R, r, l):

        # create own turtle

        self.t = turtle.Turtle()

        # set cursor shape

        self.t.shape('turtle')

        # set step in degrees

        self.step = 5

        # set drawing complete flag

        self.drawingComplete = False

        # set parameters

        self.setparams(xc, yc, col, R, r, l)
```

```
# initialize drawing

self.restart()

# set parameters

def setparams(self, xc, yc, col, R, r, l):

    # spirograph parameters

    self.xc = xc

    self.yc = yc

    self.R = int(R)

    self.r = int(r)

    self.l = 1

    self.col = col

    # reduce r/R to smallest form by dividing with GCD

    gcdVal = math.gcd(self.r, self.R)

    self.nRot = self.r//gcdVal

    # get ratio of radii

    self.k = r/float(R)

    # set color

    self.t.color(*col)

    # current angle
```

```
self.a = 0

# restart drawing

def restart(self):

    # set flag

    self.drawingComplete = False

    # show turtle

    self.t.showturtle()

    # go to first point

    self.t.up()

    R, k, l = self.R, self.k, self.l

    a = 0.0

    x = R*((1-k)*math.cos(a) + l*k*math.cos((1-k)*a/k))

    y = R*((1-k)*math.sin(a) - l*k*math.sin((1-k)*a/k))

    try:

        self.t.setpos(self.xc + x, self.yc + y)

    except:

        print("Exception, exiting.")

        exit(0)

    self.t.down()
```

```
# draw the whole thing

def draw(self):

    # draw rest of points

    R, k, l = self.R, self.k, self.l

    for i in range(0, 360*self.nRot + 1, self.step):

        a = math.radians(i)

        x = R*((1-k)*math.cos(a) + l*k*math.cos((1-k)*a/k))

        y = R*((1-k)*math.sin(a) - l*k*math.sin((1-k)*a/k))

        try:

            self.t.setpos(self.xc + x, self.yc + y)

        except:

            print("Exception, exiting.")

            exit(0)

    # done - hide turtle

    self.t.hideturtle()

    # update by one step

def update(self):

    # skip if done

    if self.drawingComplete:
```

```
    return

    # increment angle

    self.a += self.step

    # draw step

    R, k, l = self.R, self.k, self.l

    # set angle

    a = math.radians(self.a)

    x = self.R*((1-k)*math.cos(a) + l*k*math.cos((1-k)*a/k))

    y = self.R*((1-k)*math.sin(a) - l*k*math.sin((1-k)*a/k))

    try:

        self.t.setpos(self.xc + x, self.yc + y)

    except:

        print("Exception, exiting.")

        exit(0)

    # check if drawing is complete and set flag

    if self.a >= 360*self.nRot:

        self.drawingComplete = True

        # done - hide turtle

        self.t.hideturtle()
```

```
# clear everything

def clear(self):

    # pen up

    self.t.up()

    # clear turtle

    self.t.clear()

# a class for animating spiros

class SpiroAnimator:

    # constructor

    def __init__(self, N):

        # timer value in milliseconds

        self.deltaT = 10

        # get window dimensions

        self.width = turtle.window_width()

        self.height = turtle.window_height()

        # restarting

        self.restarting = False

        # create spiro objects

        self.spiros = []
```

```
for i in range(N):

    # generate random parameters

    rparams = self.genRandomParams()

    # set spiro params

    spiro = Spiro(*rparams)

    self.spiros.append(spiro)

    # call timer

    turtle.ontimer(self.update, self.deltaT)

# restart spiro drawing

def restart(self):

    # ignore restart if already in the middle of restarting

    if self.restarting:

        return

    else:

        self.restarting = True

    # restart

    for spiro in self.spiros:

        # clear

        spiro.clear()
```



```
# generate random parameters

rparams = self.genRandomParams()

# set spiro params

spiro.setparams(*rparams)

# restart drawing

spiro.restart()

# done restarting

self.restarting = False

# generate random parameters

def genRandomParams(self):

    width, height = self.width, self.height

    R = random.randint(50, min(width, height)//2)

    r = random.randint(10, 9*R//10)

    l = random.uniform(0.1, 0.9)

    xc = random.randint(-width//2, width//2)

    yc = random.randint(-height//2, height//2)

    col = (random.random(),

           random.random(),

           random.random())
```

```
    return (xc, yc, col, R, r, l)

def update(self):

    # update all spiros

    nComplete = 0

    for spiro in self.spiros:

        # update

        spiro.update()

        # count completed ones

        if spiro.drawingComplete:

            nComplete+= 1

    # if all spiros are complete, restart

    if nComplete == len(self.spiros):

        self.restart()

    # call timer

    try:

        turtle.ontimer(self.update, self.deltaT)

    except:

        print("Exception, exiting.")

        exit(0)
```

```
# toggle turtle on/off

def toggleTurtles(self):

    for spiro in self.spiros:

        if spiro.t.isvisible():

            spiro.t.hideturtle()

        else:

            spiro.t.showturtle()

# save spiros to image

def saveDrawing():

    # hide turtle

    turtle.hideturtle()

    # generate unique filename

    dateStr = (datetime.now()).strftime("%d%b%Y-%H%M%S")

    fileName = 'spiro-' + dateStr

    print('saving drawing to {}.eps/png'.format(fileName))

    # get tkinter canvas

    canvas = turtle.getcanvas()

    # save postscript image

    canvas.postscript(file = fileName + '.eps')
```

```
# use PIL to convert to PNG

img = Image.open(fileName + '.eps')

img.save(fileName + '.png', 'png')

# show turtle

turtle.showturtle()

# main() function

def main():

    # use sys.argv if needed

    print('generating spirograph...')

    # create parser

    descStr = """This program draws spirographs using the Turtle
module.

    When run with no arguments, this program draws random
    spirographs.

    Terminology:

    R: radius of outer circle.

    r: radius of inner circle.

    l: ratio of hole distance to r.

    """

    parser = argparse.ArgumentParser(description=descStr)
```

```
# add expected arguments

parser.add_argument('--sparams', nargs=3, dest='sparams',
required=False,

                        help="The three arguments in sparams: R, r, l.")

# parse args

args = parser.parse_args()

# set to 80% screen width

turtle.setup(width=0.8)

# set cursor shape

turtle.shape('turtle')

# set title

turtle.title("Spirographs!")

# add key handler for saving images

turtle.onkey(saveDrawing, "s")

# start listening

turtle.listen()

# hide main turtle cursor

turtle.hideturtle()

# check args and draw
```

```
if args.sparams:

    params = [float(x) for x in args.sparams]

    # draw spirograph with given parameters

    # black by default

    col = (0.0, 0.0, 0.0)

    spiro = Spiro(0, 0, col, *params)

    spiro.draw()

else:

    # create animator object

    spiroAnim = SpiroAnimator(4)

    # add key handler to toggle turtle cursor

    turtle.onkey(spiroAnim.toggleTurtles, "t")

    # add key handler to restart animation

    turtle.onkey(spiroAnim.restart, "space")

    # start turtle main loop

    turtle.mainloop()

# call main

if __name__ == '__main__':
```

main()
